

Dossier de Conception - Épreuve E6

TW4 - Projet Ballon Stratosphérique



Lycée
TOUCHARD
WASHINGTON

Projet réalisé par : **PAIN Iako**
BRANDS Noé
KALO Harold
GUERRIAU Lucien

Formation initiale du **BTS CIEL-IR**
2ème année à Le Mans

Projet supervisé par : **LE CREN Anthony**
KHAMLACH Jilali

Date de rendu prévue le 8 Juin 2026

Table des Matières

Table des Matières.....	2
1. Introduction.....	3
1.2. Objectifs de la Conception.....	3
2. Choix Technologiques.....	4
2.1 Matériel Embarqué (Nacelle).....	4
2.2 Logiciel Embarqué et Protocoles.....	7
2.3 Station Sol (IHM, Traitement, Données).....	8
2.4 Évaluation des Contraintes.....	9
2.5. Analyse Comparative des Outils et Justification des Décisions.....	10
2.5.1. Comparaison des Outils et Protocoles.....	10
2.5.2. Justification Globale des Décisions.....	11
3. Architecture Détaillée du Système.....	12
3.1. Structure du Logiciel.....	12
3.1.1. Modèle Modulaire.....	12
3.1.2. Synoptique du Système.....	12
Diagramme de déploiement.....	13
Analyse des Flux Principaux :.....	14
3.2. Architecture Matérielle et Logicielle.....	15
3.2.1. Équipements Matériels de la Nacelle.....	15
Description des Capteurs et Modules.....	15
Schéma des Connexions (Diagramme en Bloc).....	16
Notes sur le Câblage :.....	16
3.2.2. Flux Logiques de Traitement.....	17
3.2.3. Logiciels Associés : Frameworks, Bibliothèques et Services.....	18
B. Environnement Station Sol (Interface et Traitement).....	19
3.2.4. Contraintes et Optimisation du Système.....	20
A. Performances Attendues et Optimisation.....	20
B. Contraintes Critiques du Milieu.....	21
C. Fiabilité Énergétique.....	22
4. Structure des données et protocoles d'échange.....	23
4.1. Organisation des données manipulées :.....	23
4.2. Synthèse des choix technologiques et matérielle.....	25
ÉTUDIANT 1.....	25
1. Technologie de télémétrie.....	25
2. Technologie complémentaire de mesure de température.....	25
3. Technologie d'horodatage.....	25
4. Technologie de développement.....	25
5. Protocole de Transmission.....	26
ÉTUDIANT 2.....	27
1. Technologie de Communication : LoRa (Long Range).....	27
2. Développement de l'Interface Sol (IHM) : Framework Qt6 (C++).....	27
3. Matériel Passerelle (Gateway) : raspberry pi TTGO.....	28
4. Algorithmes de Détection : Machine à États (State Machine).....	28
ÉTUDIANT 3.....	29
1. Technologie de transmission d'image.....	29
2. Technologie de développement IHM.....	30
3. Technologie de diffusion et stockage.....	30
ÉTUDIANT 4.....	31

1. Introduction

Ce document constitue le Dossier de Conception du projet **Ballon Stratosphérique (TW4)**, réalisé par les étudiants de BTS CIEL (option Informatique et Réseaux) du Lycée Touchard-Washington.

Faisant suite au dossier d'analyse qui a défini les besoins fonctionnels et les contraintes du système, ce dossier de conception a pour but de détailler les solutions techniques retenues pour la réalisation du système embarqué et de la station sol. Il décrit l'architecture matérielle et logicielle, justifie les choix technologiques effectués (protocoles de communication, langages, composants) et définit les interfaces entre les différents modules. Ce document servira de référence technique pour la phase de développement et de validation du projet, dont le lancement est prévu pour mai 2026.

1.2. Objectifs de la Conception

L'objectif de cette phase de conception est de traduire les exigences du cahier des charges en solutions techniques concrètes et réalisables. Les objectifs spécifiques sont les suivants :

- **Valider l'Architecture Matérielle** : Confirmer l'interfaçage des capteurs (BME280, HW123) et des modules de communication (LoRa RA-02, Émetteur VHF) avec l'unité centrale.
- **Définir l'Architecture Logicielle** : Structurer les programmes embarqués (C++ sur Linux) et les applications au sol (Interface Qt/C++, Serveur Web, Base de données MariaDB) pour assurer un traitement fluide et non-bloquant des données.
- **Spécifier les Protocoles d'Échange** : Fixer le format des trames de télémétrie (APRS Weather), le protocole de transmission d'images (SSTV Martin M1) et les échanges internes (JSON, requêtes SQL).
- **Anticiper l'Intégration** : Préparer la répartition des tâches entre les quatre étudiants (Gestion IHM, Transmission LoRa, Traitement d'images, Cartographie) pour garantir une mise en commun cohérente des modules.

2. Choix Technologiques

Suite à l'analyse des besoins techniques (gestion des données, communication, interface utilisateur, etc.) et aux contraintes imposées par le contexte stratosphérique (autonomie, robustesse, portée), les choix technologiques suivants ont été arrêtés pour les systèmes embarqué et sol.

2.1 Matériel Embarqué (Nacelle)

Composant	Rôle	Justification du choix
Raspberry Pi Zero W	Micro-ordinateur	Faible consommation énergétique, encombrement minimal, environnement Linux complet pour le développement en C++, et intégration Wi-Fi/Bluetooth pour les tests au sol.
LoRa RA-02	Module de Communication Longue Portée	Permet une communication à très longue distance avec une faible consommation. Utilisé pour la transmission de la télémétrie de base.
BME280	Capteur Environnemental (Température / Humidité / Pression)	Mesure précise de la Pression, Température et Humidité. Compact et largement documenté, idéal pour les conditions atmosphériques.
LM75	Capteur Température (Température Interne)	Mesure la température interne afin de savoir si il y a une surchauffe des composants
RTC DS3231	Horodatage	Horodate les données relevées grâce à un timestamp précis.
Carte Micro SD	Stockage des données télémétrique	Idéal pour avoir une backup des données télémétrique en cas d'échec de l'envoi de celle-ci

Composant	Rôle	Justification du choix
esp32	Module de Communication Longue Portée	Permet une communication à très longue distance avec une faible consommation. Utilisé pour la transmission de requête via Aprs LoRa.
LoRa RA-02	Module de Communication Longue Portée	Permet une communication à très longue distance avec une faible consommation. Utilisé pour la transmission de la télémétrie de base.
HW123 (MPU 6050)	Capteur Mouvement (Accélération)	Mesure de l'accélération et de l'orientation pour le diagnostic des mouvements de la nacelle (choc à l'atterrissage).
SIRFstar IV	Module GPS (Position / Altitude)	Module éprouvé (ballons stratosphériques, Planète Sciences). Sa haute sensibilité , son TTFF rapide en conditions extrêmes (haute altitude, froid), sa compatibilité UART/NMEA pour intégration Linux/C++, son faible coût et sa consommation modérée justifient ce choix.

Composant	Rôle	Justification du choix
Raspberry Pi Zero W	Micro-ordinateur	Faible consommation énergétique, encombrement minimal, environnement Linux complet pour le développement en C++, et intégration Wi-Fi/Bluetooth pour les tests au sol.
Module Radioamateur 2m	Émetteur VHF	Utilisé pour la transmission des signaux SSTV (image) et comme redondance pour la télémétrie via le protocole APRS.
Pi Cam	Caméra	Permet de prendre en photo le ballon puis de mesurer la distance entre le ballon et la nacelle, le volume du ballon et vérifier la loi des gaz parfaits.
Carte Micro SD	Stockage des données images	Permet de sauvegarder les images prises par la camera dans le cas d'échec d'envoi de données

2.2 Logiciel Embarqué et Protocoles

Catégorie	Technologie Retenue	Justification du Choix
Langage de Programmation	C++ (avec environnement Linux)	Performance et contrôle bas niveau nécessaires pour l'interaction avec le matériel (SPI, I2C, UART) et l'optimisation des cycles CPU et de la consommation.
Système d'Exploitation	Raspbian Bookworm 12 Lite	Version légère de Debian optimisée pour la Raspberry Pi, offrant un environnement stable et familier.
Communication Télémétrie	LoRa + Protocole APRS	APRS est le standard radioamateur pour le suivi de position et météo, compatible avec de nombreux systèmes de réception au sol et adapté à LoRa (ou VHF).
Communication Image	SSTV (Slow Scan Television - Martin M1)	Protocole standard pour la transmission d'images en bande étroite via VHF, permettant une réception simple par la station sol.

2.3 Station Sol (IHM, Traitement, Données)

Catégorie	Technologie Retenue	Justification du Choix
Interface Utilisateur (IHM)	Qt / C++ 	Framework cross-platform robuste, idéal pour la création d'une application de bureau performante assurant le traitement temps réel des données et l'affichage cartographique.
Gestion de Base de Données	MariaDB (ou MySQL) 	Base de données relationnelle éprouvée, nécessaire pour stocker historiquement les trames de télémétrie et les événements de vol (altitude maximale, heure de largage, etc.).
Cartographie	OSM 	Solution open source permettant l'affichage dynamique de la position GPS du ballon sur une carte web intégrée à l'IHM.
Communication Réception	SDR (Software Defined Radio) 	Permet la réception des signaux LoRa et VHF/APRS/SSTV, avec la flexibilité de traitement logiciel.

Ces choix ont été faits en tenant compte de la balance entre performance technique, fiabilité requise par l'environnement stratosphérique, et les compétences des étudiants dans le cadre du cursus BTS CIEL.

2.4 Évaluation des Contraintes

Les choix technologiques présentés ci-dessus ont été évalués par rapport à un ensemble de contraintes critiques pour la réussite du projet Ballon Stratosphérique.

Contrainte	Évaluation et Justification des Choix
Performances du Système	Système Embarqué : Le choix du C++ sur Raspberry Pi Zero W assure les performances nécessaires pour l'acquisition et le traitement temps réel des données capteurs et la génération des signaux de transmission (LoRa, SSTV) sans surcharge excessive du CPU. Le système d'exploitation allégé (Raspbian Lite) minimise les latences. Station Sol : L'utilisation de Qt/C++ garantit une IHM réactive, essentielle pour le suivi en temps réel et la cartographie dynamique.
Compatibilité Matérielle	Tous les composants matériels (BME280, SIRFstar IV , LoRa RA-02) ont été choisis pour leur compatibilité éprouvée avec l'environnement Linux et les interfaces de communication standard de la Raspberry Pi (I2C, SPI, UART). Les bibliothèques associées sont disponibles et stables, réduisant les risques d'intégration.
Portabilité	Système Embarqué : Le code C++ est développé pour être intrinsèquement portable. Cependant, l'interaction directe avec les GPIO et les bus de la Raspberry Pi Zero W limite la portabilité logicielle à d'autres plateformes non-Linux ou sans architecture ARM similaire. Station Sol : Le framework Qt est nativement multiplateforme (Windows, Linux, macOS), assurant une excellente portabilité de l'IHM pour l'équipe de récupération et d'exploitation.
Coût Global	Le Raspberry Pi Zero W et les capteurs choisis sont des solutions à faible coût, respectant le budget contraint du projet éducatif. Le matériel de communication (LoRa, Module VHF radioamateur) est également accessible, permettant de dédier la majeure partie du budget aux tests et à la robustesse de l'enveloppe et du parachute.

2.5. Analyse Comparative des Outils et Justification des Décisions

Le choix des technologies n'a pas été arbitraire, mais le fruit d'une analyse comparative rigoureuse des solutions disponibles (frameworks, bibliothèques, protocoles) afin d'assurer la meilleure adéquation entre les exigences du projet et les contraintes techniques et pédagogiques.

2.5.1. Comparaison des Outils et Protocoles

Domaine	Outils/Protocoles Comparés	Outil Retenu	Justification de la Décision
IHM Station Sol	JavaFX, Electron, Qt/C++	Qt / C++	Qt offre le meilleur compromis entre performance (proximité C++), robustesse pour une application critique, et capacités graphiques pour l'intégration de la cartographie (vs. JavaFX moins performant, Electron trop lourd/consommateur)
Base de Données	PostgreSQL, MongoDB (NoSQL), MariaDB/MySQL	MariaDB/MySQL	Base de données relationnelle standard, facile à déployer et bien documentée, essentielle pour le stockage structuré de données de télémétrie précises. La simplicité de mise en œuvre est privilégiée.
Transmission Télémétrie	Protocole propriétaire, MQTT, APRS	APRS	APRS est un standard radioamateur reconnu pour le suivi de ballons. Il est optimisé pour les transmissions de données courtes et intermittentes et assure une compatibilité avec les réseaux de suivi mondiaux (fiabilité et redondance accrues).
Transmission Image	RAW/JPEG compressé, WEFAX, SSTV (Martin M1)	SSTV (Martin M1)	Le SSTV est conçu pour la transmission d'images via bande étroite (VHF). Le mode Martin M1 est couramment utilisé pour sa rapidité relative et sa bonne résilience au bruit radio. Il minimise la bande passante requise.
Langage Embarqué	Python, C/C++ , Go	C++	Nécessité de contrôle bas niveau (GPIO, I2C, SPI) et d'optimisation de la performance/consommation, que le C++ assure mieux que Python (interprété) ou Go (moins de contrôle bas niveau sur RPi Zero).

2.5.2. Justification Globale des Décisions

Les décisions prises sont justifiées par leur pertinence vis-à-vis des objectifs et leur capacité à répondre aux exigences du système, tout en anticipant sa fiabilité, son évolutivité, et sa maintenabilité :

- **Fiabilité et Robustesse** : Le choix de technologies éprouvées dans le domaine de la haute altitude (APRS, SSTV, BME280) et de langages performants (C++) minimise les risques de défaillance en vol. La simplicité de Raspbian Lite contribue également à la stabilité du système embarqué.
- **Évolutivité** : L'utilisation de protocoles standardisés (APRS, bases de données relationnelles) et d'un framework modulaire (Qt) permet d'intégrer facilement de nouveaux capteurs ou de nouvelles fonctionnalités (par exemple, un contrôle à distance plus sophistiqué) sans refonte majeure du système.
- **Maintenabilité et Pédagogie** : Les outils retenus (Qt, C++, MariaDB) sont alignés avec les compétences développées dans le cursus BTS CIEL. Cela facilite la documentation, la correction des bugs et l'appropriation du code par l'équipe, tout en offrant une courbe d'apprentissage pertinente pour leur formation.
- **Efficacité Énergétique** : Le choix de la raspberry pi et du C++ pour l'embarqué est essentiel pour maximiser l'autonomie de la nacelle, une contrainte majeure pour un vol de longue durée.

3. Architecture Détaillée du Système

3.1. Structure du Logiciel

La structure du logiciel embarqué est conçue selon une architecture modulaire, visant à séparer clairement les responsabilités (acquisition, traitement, communication) afin d'assurer la robustesse et la maintenabilité du code. Le système est basé sur un schéma de boucles d'événements non bloquantes, essentielles pour gérer simultanément l'acquisition des données, la gestion des états de vol et la transmission radio.

3.1.1. Modèle Modulaire

Le logiciel sera décomposé en modules principaux communiquant via des interfaces bien définies :

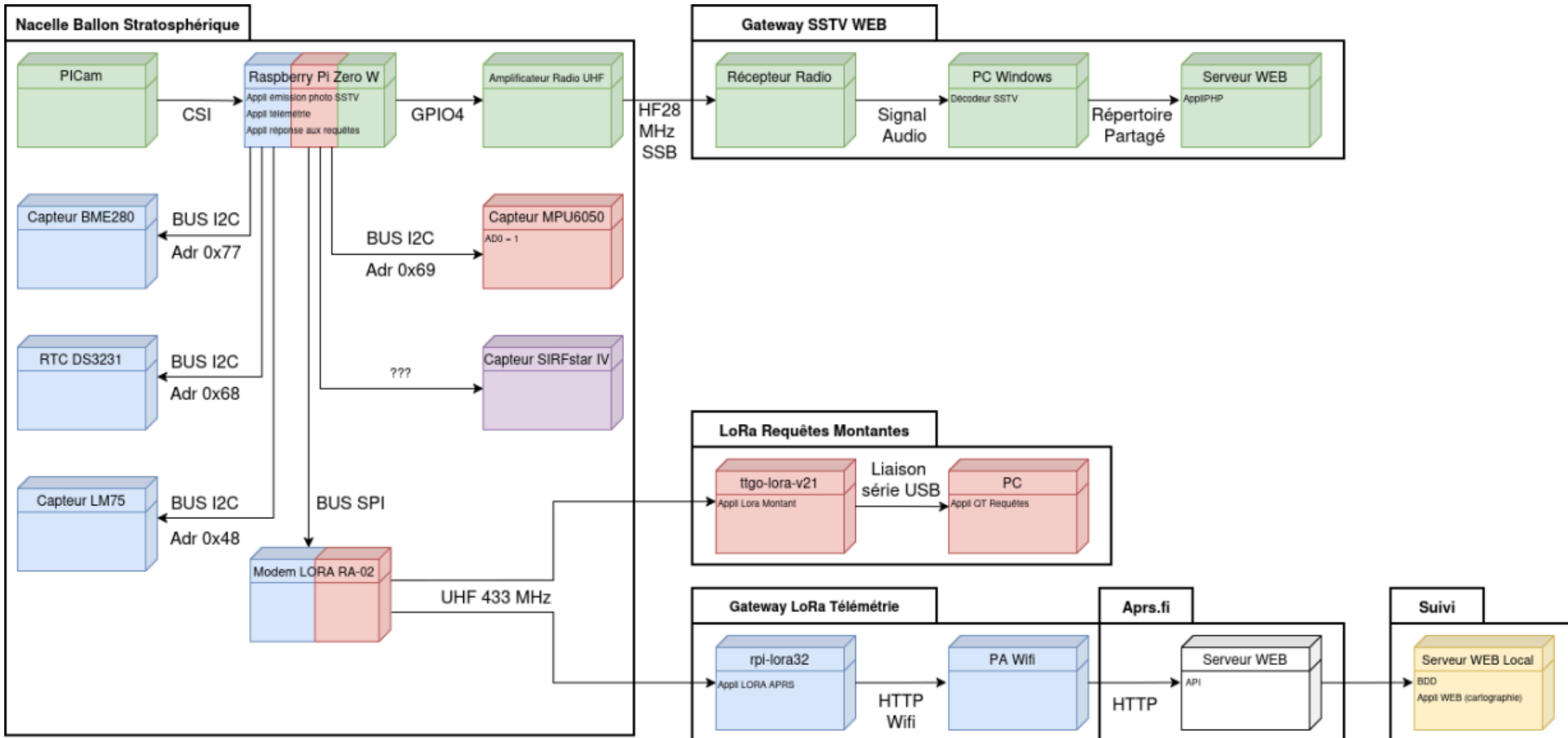
1. **Module d'Acquisition** : Gère l'initialisation et la lecture des capteurs (BME280, HW123, SRFstar IV).
2. **Module de Traitement** : Effectue le filtrage, la mise en forme (standardisation des unités) et la structuration des données de télémétrie.
3. **Module de Communication** : Responsable de l'encodage (APRS, SSTV) et de la transmission physique (LoRa, VHF).
4. **Module de Logique de vol (Main Loop)** : Le cœur du système. Gère les états de vol (Préparation, Montée, Stabilité, Descente/Largage) et orchestre les interactions entre les autres modules.

3.1.2. Synoptique du Système

Pour illustrer les relations entre les composants matériels et les sous-systèmes logiciels, un diagramme de déploiement (basé sur le standard UML) est présenté ci-dessous. Il met en évidence la répartition des tâches sur le nœud embarqué (Nacelle) et le nœud au sol (Station Sol), ainsi que les flux de données et d'énergie.

Élément	Description
Nœud : Nacelle (raspberry pi)	Composants système : Capteurs, GPS, Batterie.
Nœud : Station Sol (Ordinateur de bord)	IHM et base de données : Affichage des données, Stockage, Traitement.
Artefacts	Logiciels ou protocoles clés : Communication LoRa, Logiciel Python, SQLite.
Associations	Relations de communication ou d'alimentation : Liaison LoRa (Nacelle ↔ Station Sol), Alimentation (Batterie ← Composants Nacelle).

Diagramme de déploiement



Analyse des Flux Principaux :

Flux	Description	Direction	Support/Protocole
Flux de Données - Télémétrie	Acquisition des données capteurs et transmission au sol pour le suivi.	Nacelle -> Station Sol	Bus I2C/UART (interne) ; LoRa + APRS (radio)
Flux de Données - Image (SSTV)	Envoi des images capturées par la caméra au sol pour décodage.	Nacelle -> Station Sol	Bus USB/GPIO (interne) ; VHF + SSTV (radio)
Flux de Données - Enregistrement	Stockage local des données de télémétrie	Interne Nacelle	Carte SD (Fichier Log)
Flux de Contrôle - IHM	Commandes envoyées par l'opérateur (hypothétique, pour des évolutions futures).	Station Sol -> Nacelle	LoRa (bidirectionnel) ou VHF (redondance)
Flux d'Énergie	Alimentation du système embarqué.	Batterie -> Nacelle	Câblage électrique

3.2. Architecture Matérielle et Logicielle

3.2.1. Équipements Matériels de la Nacelle

L'architecture matérielle de la nacelle est centrée autour de l'unité de traitement (raspberry pi) et des différents modules d'acquisition et de communication. L'ensemble est conçu pour minimiser le poids, la consommation énergétique et maximiser la résilience aux basses températures.

<i>Description des Capteurs et Modules</i>			
Composant	Type/Rôle	Interface de Communication	Description
raspberry pi	Concentrateur / Unité de traitement	SPI, I2C, UART, GPIO	Cœur du système embarqué, exécute le logiciel C++ et gère les communications.
BME280	Capteur Environnemental	I2C	Mesure de la Pression atmosphérique, de la Température ambiante et de l'Humidité. Crucial pour les données de télémétrie.
HW123 (MPU 6050)	Capteur de Mouvement (IMU)	I2C	Accéléromètre et Gyroscope. Utilisé pour détecter l'orientation et l'impact à l'atterrissage.
RTC DS3231	Module d'horodatage	I2C	Horodate des données.
Capteur LM75	Module de température interne	I2C	Mesure la température interne au système.
SIRFstar IV	Capteur de Position (GPS)	UART (Série)	Fournit la position (latitude, longitude, altitude) . Les données brutes sont transmises au module de communication.
LoRa RA-02	Module de Communication Radio	SPI	Transmetteur à faible consommation pour la télémétrie APRS principale sur longue distance.
Module Radioamateur 2m (VHF)	Module de Communication Radio	GPIO	Utilisé pour la transmission des images SSTV et comme lien de télémétrie APRS redondant.
Caméra Pi TNBA1392	Actionneur / Acquisition Image	CSI	Capture les images pour la transmission SSTV (ralentie pour bande étroite).

Schéma des Connexions (Diagramme en Bloc)

Le schéma ci-dessous illustre les interconnexions physiques des principaux composants sur la carte raspberry pi. L'accent est mis sur l'utilisation des bus numériques standard pour l'acquisition des données et des GPIO pour le contrôle des modules radio.

mettre

Notes sur le Câblage :

- **I2C Bus** : Le BME280 et le HW123 partagent le bus I2C (SDA/SCL) de la raspberry pi, minimisant le nombre de broches utilisées.
- **UART** : Le module GPS utilise le port série (UART) pour envoyer les trames NMEA à l'unité centrale.
- **SPI** : Le module LoRa utilise le bus SPI (MOSI, MISO, SCLK, CE) pour un transfert de données rapide et efficace.
- **Alimentation** : Tous les modules sont alimentés par la même source de batterie via un circuit de régulation de tension adapté, garantissant 3.3V ou 5V selon les besoins du composant

3.2.2. Flux Logiques de Traitement

Le système logiciel embarqué est structuré autour d'un ensemble de tâches concurrentes orchestrées par le *Module de Logique de Vol* (Main Loop).

Tâche	Fréquence d'Exécution	Interfaces Logiciel/Matériel	Dépendances
Acquisition Capteurs	1 Hz (toutes les secondes)	I2C (BME280,HW123), UART (SIRFstar IV)	Aucune
Traitement Télémétrie	1 Hz	Module de Logique de Vol	Données GPS (position), Données Météo (BME280)
Transmission LoRa (APRS)	1 message / 60 secondes	SPI (RA-02)	Télémétrie traitée
Prise de Vue	1 image / 5 minutes	CSI (Pi-Cam)	Logique de Vol (déclenchement par altitude ou temps)
Transmission SSTV (VHF)	Après chaque Prise de Vue	GPIO	Image capturée
Gestion des Logs (Carte SD)	1 Hz	Système de Fichiers	Télémétrie brute et traitée

Cette architecture garantit que l'échec ou le blocage d'un module (par exemple, échec d'une transmission radio) n'interrompt pas les fonctions critiques telles que l'acquisition des données ou l'enregistrement local (logging).

3.2.3. Logiciels Associés : Frameworks, Bibliothèques et Services

Le choix des frameworks, des bibliothèques et des services constitue une étape cruciale pour garantir la robustesse, la performance et l'efficacité du système dans son ensemble. L'architecture logicielle a été conçue pour optimiser le temps de développement tout en s'appuyant sur des solutions éprouvées, spécifiquement adaptées aux contraintes de l'environnement embarqué (nacelle) et de l'interface utilisateur (station sol).A.
Environnement Embarqué (Nacelle)

Le logiciel embarqué, développé principalement en C++ pour ses performances et son contrôle bas niveau sur le matériel, utilise des bibliothèques essentielles pour l'acquisition de données et la communication :

Catégorie	Technologie Retenue	Rôle Spécifique	Justification Détaillée
Gestion des Interfaces Périphériques (GPIO)	WiringPi/pigpio (ou équivalent)	Abstraction et gestion des communications bas niveau (I2C, SPI, UART, GPIO) sur la Raspberry Pi (RPI). Ces interfaces sont indispensables pour dialoguer avec les capteurs et le module de transmission.	La RPi, bien que puissante, nécessite des bibliothèques spécifiques pour interagir efficacement avec ses broches. Ces outils offrent une API simplifiée et des performances suffisantes pour la lecture synchrone et asynchrone des données capteurs. <i>Note : Ces bibliothèques sont privilégiées pour leur légèreté et leur orientation bas-niveau, contrairement à des solutions d'OS plus lourdes.</i>
Traitement GPS	TinyGPS++	Analyse syntaxique et traitement des trames NMEA brutes reçues du module GPS (U-blox NEO-M8N). Extraction rapide de la position (latitude, longitude), de l'altitude, de l'heure et de la vitesse.	Cette bibliothèque se distingue par sa petite taille et son efficacité de traitement, des qualités essentielles dans un environnement embarqué où les ressources CPU et mémoire sont limitées. Elle assure un décodage fiable même en cas de trames incomplètes ou corrompues.
Interface Capteurs	Bibliothèques constructeurs (BME280, HW123)	Initialisation, configuration et lecture des données des capteurs environnementaux (pression, température, humidité) et de l'unité de mesure inertielle (IMU - accélération, rotation).	L'utilisation des bibliothèques officielles ou de versions communautaires largement supportées minimise les risques d'erreur de mesure et garantit l'alignement avec les spécifications techniques des fabricants. Cela assure la précision et la fiabilité des mesures télémétriques.

B. Environnement Station Sol (Interface et Traitement)

La station sol est responsable de la réception, de la visualisation en temps réel et de l'archivage des données. L'utilisation de technologies multiplateformes comme Qt et des outils de traitement de données performants comme Python a été choisie pour ces objectifs:

Catégorie	Technologie Retenue	Rôle Spécifique	Justification Détaillée
Interface Homme-Machine (IHM)	Qt Widgets	Fourniture de l'ossature de l'interface graphique utilisateur (fenêtres, menus, disposition des éléments). Assure une ergonomie de base pour la surveillance des paramètres de vol.	Qt est un framework C++ mature et multiplateforme, permettant de développer une application autonome et performante pour la station sol. L'approche basée sur les <i>Widgets</i> offre une grande flexibilité et un contrôle précis sur l'interface.
Visualisation Dynamique	Qt Charts	Affichage graphique dynamique et en temps réel des données de télémétrie (altitude vs. temps, température vs. temps, vitesse verticale).	Crucial pour l'analyse immédiate du profil de vol. Ces modules spécialisés dans la génération de graphiques offrent une fluidité et une réactivité indispensables pour traiter un flux constant de données reçues par radio. QCustomPlot est souvent considéré pour sa simplicité et sa légèreté.
Traitement du Signal et de l'Image	Logiciel SSTV : YONIQ	Transformer la matrice de pixels (RGB) de ta photo capturée par la Camera Pi en une suite de fréquences audio modulées selon le timing très précis du mode Martin M1.	Le mode Martin M1 demande une précision à la microseconde pour que l'image ne soit pas "penchée" à la réception. Yoniq est optimisé pour respecter ces timings rigoureux, garantissant une image parfaitement droite et stable au sol.
Gestion de Base de Données (SGBD)	Connecteur MariaDB pour C++/Python	Connexion sécurisée et exécution des requêtes SQL pour l'enregistrement persistant des données de télémétrie et des logs de mission, ainsi que pour la lecture des données historiques.	MariaDB a été sélectionnée comme solution SGBD pour sa robustesse, sa performance et sa licence open-source. Le connecteur assure l'interface nécessaire entre l'application de l'IHM et la base de données, garantissant l'intégrité et la persistance de toutes les données collectées durant la mission.

3.2.4. Contraintes et Optimisation du Système

Les choix technologiques et les architectures logicielles détaillées précédemment sont le résultat d'une évaluation rigoureuse des contraintes techniques inhérentes à un projet de ballon stratosphérique. La réussite de la mission repose sur la capacité du système à opérer de manière fiable dans un environnement hostile, avec des ressources limitées.

A. Performances Attendues et Optimisation

Le système doit répondre à des performances minimales pour garantir la collecte et la transmission de données critiques :

Indicateur de Performance	Objectif Cible	Justification
Fréquence d'Acquisition Capteurs	1 Hz (1 lecture/seconde)	Nécessaire pour capturer avec précision les variations rapides de l'altitude (taux de montée) et des paramètres environnementaux.
Délai de Traitement Télémétrie	< 500 ms	Le temps entre l'acquisition des données GPS/Capteurs et la mise en trame APRS doit être minimal pour garantir la fraîcheur de l'information.
Débit de Transmission LoRa	Minimal (quelques octets)	Optimisation de la durée d'émission et de la consommation. Le protocole APRS est conçu pour de faibles débits.
Temps de Traitement SSTV	< 5 minutes/image	Compte tenu de la durée du vol (plusieurs heures), la transmission d'une image doit rester inférieure à l'intervalle de prise de vue.
Temps de Réponse IHM (Station Sol)	< 200 ms	Essentiel pour une visualisation temps réel fluide et une prise de décision rapide par l'équipe au sol (par exemple, lors du suivi de l'atterrissage).

B. Contraintes Critiques du Milieu

Le milieu stratosphérique impose des contraintes physiques directes qui ont orienté les choix de conception :

Contrainte Milieu	Impact sur le Système	Mesures d'Atténuation/Solutions
Basse Température	Diminution de l'efficacité des batteries, risque de dysfonctionnement des composants électroniques.	Utilisation de composants de qualité industrielle/automobile (si possible), confinement thermique (boîtier isolé), et sélection de la raspberry pi qui dégage peu de chaleur.
Basse Pression	Risque de décharges corona (à haute altitude) et de problèmes de refroidissement (faible densité de l'air).	Conception mécanique assurant l'étanchéité ou le fonctionnement dans le vide pour l'électronique sensible. Vérification des spécifications des capteurs (BME280 est adapté à de très faibles pressions).
Rayonnements	Augmentation des risques d'erreurs logicielles (bit flip) ou de dégradation des composants.	Utilisation d'un système d'exploitation allégé et stable (Raspbian Lite) et de code C++ robuste avec gestion des erreurs (CRC pour les données critiques).
Vibrations/Choc	Risque de déconnexion des modules lors du largage du ballon et à l'atterrissage.	Conception mécanique robuste du boîtier et utilisation de connecteurs soudés ou de nappes sécurisées plutôt que de simples contacts (e.g., <i>breadboards</i>).

4. Structure des données et protocoles d'échange

4.1. Organisation des données manipulées :

```
{
  "command": "get",
  "result": "ok",
  "what": "loc",
  "found": 1,
  "entries": [
    {
      "name": "OH7RDA",
      "type": "I",
      "time": "1267445689",
      "lasttime": "1270580127",
      "lat": "63.06717",
      "lng": "27.66050",
      "symbol": "V#",
      "srcall": "OH7RDA",
      "dstcall": "APND12",
      "phg": "44603",
      "comment": "VR,W,Wn,Tn Siilinjarvi",
      "path": "WIDE2-2,qAR,OH7AA"
    }
  ]
}
```

Dictionnaire des données : Tableau décrivant chaque champ de donnée

(nom, type, contrainte).

		Sécurité et intégrité : Plan de sauvegarde, chiffrement et gestion des accès.		
		Plan de sauvegarde	Chiffrement	Gestion des accès
É1	Carte SD	Sauvegarde sous format .json sur la carte SD	Non requis	Non requis
É2	Non Requis	Non requis	Vérification d'Intégrité des Messages sur les trames de commande LoRa	Non requis
É3	Carte SD	Sauvegarde sous format .jpeg ou jpg sur la carte SD	Non requis	Non requis
É4	BDD et Site Web	Automatique	HTTPS	Compte Admin (MDP)

4.2. Synthèse des choix technologiques et matérielle

ÉTUDIANT 1

Domaine : Communication Radio, Systèmes Embarqués & Interface Homme-Machine (IHM)

1. Technologie de télémessure

Choix retenu : BME280 (Adr 0x77)

Justification :

- **Pluralité des données (3-en-1)** : Ce module permet le relevé simultané de la température, de l'humidité et de la pression atmosphérique. Cette polyvalence réduit l'encombrement dans la nacelle tout en offrant une fonction altimètre de précision (± 1 hPa)
- **Consommation** : Le capteur est extrêmement économe (environ 3.6 μ A en fonctionnement), ce qui est critique pour préserver l'autonomie de la batterie durant les 3 à 4 heures de vol.
- **Fiabilité** : Contrairement aux modèles basiques, le BME280 offre une excellente stabilité face aux variations thermiques brutales de la stratosphère, garantissant des mesures cohérentes tout au long de la mission.

2. Technologie complémentaire de mesure de température

Choix retenu : LM75 (Adr 0x48)

Justification:

- **Précision** : Le capteur peut mesurer une température plus chaude ou froide que le BME280, allant donc de -55°C à $+125^{\circ}\text{C}$.
- **Dualité de température** : Le LM75 permet de mesurer la température ambiante externe mais aussi la température interne du système, très utile donc pour savoir si celui-ci surchauffe

3. Technologie d'horodatage

Choix retenu : RTC DS3231 (Adr 0x69)

Justification:

- **Stabilité thermique** : Contrairement aux quartz classiques, son oscillateur compensé en température (TCXO) reste précis malgré le froid extrême de la stratosphère, évitant tout décalage temporel.
- **Autonomie** : Grâce à sa pile de secours, il maintient l'heure exacte même hors tension, assurant un horodatage immédiat dès le démarrage.

4. Technologie de développement

Choix retenu : IDE NetBeans (Apache)

Justification :

- **Gestion de projet intégrée** : NetBeans offre une interface robuste pour la gestion du code source, facilitant la compilation et le déploiement sur différentes plateformes.

Ses outils de débogage avancés permettent d'identifier rapidement les erreurs de logique avant le lancement.

- **Polyvalence et Extensibilité** : Bien que réputé pour le Java, il supporte efficacement le C/C++ et d'autres langages via des plugins. Cela permet de centraliser le développement de la station au sol et des scripts d'analyse de données dans un seul environnement.
- **Open Source et Ergonomie** : C'est un outil gratuit, multiplateforme et doté d'une complétion de code intelligente qui accélère l'écriture des algorithmes de traitement des données télémétriques.

5. Protocole de Transmission

Choix retenu : Modem LORA RA-02 (Bus SPI)

Justification :

- **Standardisation des données (APRS)** : L'APRS (*Automatic Packet Reporting System*) est le standard mondial pour le partage de données télémétriques et météo. En l'utilisant, les données de la sonde (température, pression, position) sont structurées de manière universelle, facilitant leur traitement par les logiciels au sol.
- **Performance du lien (LoRa)** : Contrairement à l'APRS classique (VHF) qui nécessite du matériel lourd et énergivore, la modulation LoRa offre une sensibilité extrême (**-148 dBm**). Cela permet de recevoir les paquets de données à plus de **100 km** avec une puissance d'émission très faible (25 mW).
- **Infrastructure existante** : Le choix du LoRa-APRS permet de bénéficier du réseau mondial de passerelles (**IGates**). La nacelle est captée par une passerelle amateur au sol, vos données météo sont automatiquement renvoyées sur internet (via *aprs.fi*), sécurisant ainsi la récupération des données en cas de perte du lien direct.

ÉTUDIANT 2

Domaine : Communication Radio, Systèmes Embarqués & Interface Homme-Machine (IHM)

1. Technologie de Communication : LoRa (Long Range)

Choix retenu : Modulation LoRa (433 MHz) via module RA-02 (SX1278) sur la nacelle et module raspberry pi TTGO (SX1276) au sol.

Justification :

- **Portée et Bilan de liaison :** Contrairement au Wi-Fi ou au Bluetooth (portée trop courte) ou aux réseaux cellulaires 4G (inexistants en haute altitude), la technologie LoRa offre une sensibilité de réception exceptionnelle (-148 dBm), indispensable pour maintenir le lien avec le ballon stratosphérique sur plusieurs dizaines de kilomètres.
- **Consommation :** La modulation LoRa est très peu énergivore, ce qui est critique pour l'autonomie sur batterie de la nacelle durant les 3 à 4 heures de vol.
- **Interopérabilité :** Le choix de la fréquence 433 MHz et du protocole APRS (trames AX.25) permet non seulement de communiquer avec notre station sol, mais aussi d'être relayé par le réseau des radioamateurs et suivi via le site *aprs.fi*.

2. Développement de l'Interface Sol (IHM) : Framework Qt6 (C++)

Choix retenu : Développement d'une application bureau en C++ avec les bibliothèques Qt6 (Qt Serial Port, Qt Widgets).

Justification :

- **Gestion du Matériel (Port Série) :** Le besoin critique de mon interface est de communiquer de manière fiable via USB avec la Gateway LoRa (raspberry pi) La classe `QSerialPort` de Qt offre une abstraction robuste et performante pour gérer ces échanges asynchrones (lecture/écriture de trames), supérieure à une gestion de fichiers classique.
- **Performance et Robustesse :** Contrairement à une interface Web ou Python qui pourrait souffrir de latences, le C++ compilé garantit une réactivité immédiate lors de l'envoi de commandes critiques (ex: requêtes RSSI/SNR) ou de la réception d'alertes (Chute libre).
- **Portabilité et Pérennité :** Qt est un standard industriel multiplateforme (Windows/Linux), assurant que l'outil de contrôle pourra être déployé sur n'importe quel PC de la station sol sans réécriture du code.

3. Matériel Passerelle (Gateway) : raspberry pi TTGO

Choix retenu : Microcontrôleur rpi lora32 avec module LoRa intégré (carte TTGO).

Justification :

- **Intégration** : Ce module "tout-en-un" évite un câblage complexe entre un microcontrôleur et un module radio séparé, réduisant les risques de pannes matérielles au sol.
- **Puissance de traitement** : La raspberry pi dispose de deux cœurs, permettant de gérer la pile LoRa sur un cœur et la communication Série USB vers le PC sur l'autre, évitant tout goulot d'étranglement lors de la réception de rafales de données télémétriques.
- **Environnement de développement** : Compatible avec l'IDE PlatformIO/Arduino (C++), il permet de réutiliser une partie du code driver LoRa développé pour la nacelle, optimisant le temps de développement.

4. Algorithmes de Détection : Machine à États (State Machine)

Choix retenu : Implémentation d'une machine à états finis en C++ pour la détection de chute et d'atterrissage.

Justification :

- **Fiabilité** : Pour détecter l'éclatement du ballon (passage en chute libre) ou l'atterrissage, la simple lecture de seuils d'accélération (MPU6050) ne suffit pas car elle est sujette au bruit. Une structure en "Machine à États" (Ascension -> Chute -> Atterrissage) permet de valider des transitions logiques et d'éviter les faux positifs (ex: secousse lors du lancement interprétée comme un atterrissage) .

ÉTUDIANT 3

Domaine : Transmission d'images (SSTV), Développement Web & Traitement Vidéo

1. Technologie de transmission d'image

Choix retenu : Protocole SSTV (Mode Martin M1)

Justification :

- **Qualité d'image supérieure :** Le mode Martin M1 est un standard européen réputé pour sa fidélité des couleurs et sa netteté. Contrairement aux modes plus rapides, il décompose chaque ligne de l'image avec une précision accrue, ce qui permet d'obtenir des clichés de la stratosphère très détaillés.
- **Robustesse du signal :** En transformant les pixels en fréquences audio, le SSTV Martin M1 reste parfaitement décodable même si le signal radio subit des affaiblissements dus à la distance ou à l'inclinaison du ballon. L'image peut être parasitée, mais elle reste entière et exploitable.
- **Synchronisation fiable :** Le protocole Martin utilise des impulsions de synchronisation spécifiques pour chaque ligne, ce qui garantit que l'image ne se "décalera" pas horizontalement pendant la réception, assurant ainsi une galerie photo propre pour le public au sol.

Autres choix possibles : Protocole SSTV (Mode Scottie S1 et Robot 36)

Contraintes temporelles :

- Le protocole Robot 36 est très rapide (36s), mais il laisserait la radio inactive pendant plus de 4 minutes. C'est un manque d'efficacité pour une mission de 4h où l'on veut un flux constant. Tandis que le protocole Martin M1, occupe bien le canal radio sans le saturer avec environ 2 minutes de transmission.

Contraintes matérielles :

- Ces deux modes sont très proches en durée. Cependant, le Martin M1 est le standard européen par excellence. Il possède une gestion de la chrominance (couleurs) plus stable. Puis on utilise une CameraPi. Si on utilise le protocole Robot 36, cela gâcherait la qualité optique du capteur.

2. Technologie de développement IHM

Choix retenu : IDE NetBeans (Apache)

Justification :

- **Conception d'interface:** NetBeans intègre un éditeur visuel puissant qui facilite la création de l'interface graphique de la station sol. Cela permet de disposer de boutons de contrôle, de fenêtres de visualisation d'images et d'indicateurs d'état de connexion clairs pour l'opérateur.
- **Fiabilité du code :** Grâce à ses outils de débogage et de gestion de projet, NetBeans permet de structurer proprement le logiciel de décodage SSTV et de s'assurer que le traitement du signal audio ne subit pas de ralentissements critiques.
- **Intégration multi-langages :** Il permet de centraliser le développement des outils de contrôle et des scripts de gestion de données dans un environnement professionnel unique, facilitant la maintenance du projet.

3. Technologie de diffusion et stockage

Choix retenu : Architecture Web (PHP / MariaDB)

Justification :

- **Performance et Open Source (MariaDB) :** MariaDB est une base de données relationnelle optimisée, idéale pour fonctionner sur des systèmes légers. Elle permet de stocker l'index des photos (noms, dates, altitudes) de manière sécurisée et très rapide.
- **Accessibilité universelle :** Le couplage avec PHP permet de générer une galerie dynamique. Les utilisateurs peuvent consulter les images du vol depuis n'importe quel appareil (smartphone, PC) sans avoir à installer de logiciel spécifique.
- **Évolutivité :** Cette structure permet d'ajouter facilement de nouvelles fonctionnalités au site, comme un tri par heure ou une visualisation des statistiques de réception, rendant la mission interactive pour le public.

ÉTUDIANT 4

Choix de matériel:

BDD: MariaDB

Logiciel de programmation: QT & Netbeans (imposé)

Serveur: PC du lycée apache (imposé)

Base	Type	Cas d'usage
PostgreSQL	SQL	Général / complexe
<u>MySQL / MariaDB</u>	SQL	Webs apps classiques
SQLite	SQL	Mobile / prototypes
MongoDB	NoSQL	JSON docs / flexibilité
Redis	NoSQL	Cache / sessions
Cassandra / Scylla	NoSQL	Big Data distribué

Critère	Pourquoi QT est adapté ?
Gestion HTTP native	QNetworkAccessManager
Support JSON	QJsonDocument/Object/Array
Gestion des BDD	QSqlDatabase/Query
Multiplate-forme	Windows, Linux, macOS
Gestion SSL	Support HTTPS
Gestion des données entrent	WebSocket avec QTCPSocket

APRS Weather (Envoi des données télémétrique du ballon)

Champ	Longueur	Signification	Exemple
Source	Variable (5 à 10)	Indicatif Radio du ballon	F4KMN-9
Séparateur	1	Sépare la source du format	>
Format	Variable (3 à 6)	Indique le format utilisé	APRS
Séparateur	1	Sépare le format du chemin	,
Chemin	Variable (5 à 10)	Chemin utilisé, par exemple par un répéteur	WIDE1-1
Séparateur	1	Sépare les indicateurs de source des données d'horodatage et télémétrie	:
Indicateur	1	Indique le début du timestamp	_
Timestamp	7	Donne le mois , le jour , l'heure et la minute (format MMJJHHmm)	09120556
Non utilisé	12	Relatif à la direction (c) , la vitesse (s) et l'accélération (g) du vent	c...s...g...
Indicateur	1	Indicateur de température	t
Température	3	Température, encodage décimal en °F	081
Indicateur	1	Indicateur d'humidité	h
Humidité	3	Humidité, encodage décimal en %	58
Indicateur	1	Indicateur de pression (b comme baromètre)	b
Pression	5	Pression, encodage binaire traduit en float	10001
EXEMPLE TRAME	F6HTB>APU25N,UNCAN*:@091842z4256.20N07049.42W_t081h58b10001/X0.15Y-0.02Z9.81		

LoRa requête		
Élément	Symbole	Rôle dans le programme
Source	callsign	Ton indicatif émetteur (ex: F4KMN-9).
Séparateur	>	Indique la direction du flux.
Destination	destination	Code de destination APRS (ex: APLT00 pour LoRa).
Chemin	,path	Chemin de répétition (ex: ,WIDE1-1).
Type de paquet	::	Le double "deux-points" annonce un paquet de type APRS Message.
Recipient	recipient	Le destinataire final (ex: F4KMN). Doit faire 9 caractères.
Séparateur	:	Sépare le destinataire du contenu.
Contenu	comment	Le texte de ta requête. Limité à 67 caractères par ton code.
Accusé (opt)	{	Présent uniquement si ack est à true .
ID (opt)	messageld	Identifiant du message pour l'ACK (0 à 99999).
Exemple complet : F4KMN-9>APLT00,WIDE1-1::F4KMN :Hello{1		

SSTV			
Champ	Longueur	Signification	Exemple
Header (VIS)	300 ms	Signal d'étalonnage pour indiquer le début de l'image.	1900 Hz
Sync Périodique	4.87 ms	Impulsion de synchronisation au début de chaque ligne.	1200 Hz
Porch	0.57 ms	Petit silence de séparation avant les couleurs.	1500 Hz
Scan Vert (G)	146.43 ms	Fréquence variant selon l'intensité du vert du pixel.	1500 - 2300 Hz
Scan Bleu (B)	146.43 ms	Fréquence variant selon l'intensité du bleu du pixel.	1500 - 2300 Hz
Scan Rouge (R)	146.43 ms	Fréquence variant selon l'intensité du rouge du pixel.	1500 - 2300 Hz
Sync Fin de ligne	4,87 ms	Signal indiquant que la ligne est terminée.	1200 Hz
EXEMPLE TRAME	[1200Hz / 4,87ms] - [1500Hz / 0,57ms] - [1750Hz / 146ms] - [2100Hz / 146ms] - [1900Hz / 146ms]		

Étape	Résultat
1	Création d'un compte APRS.fi sur https://aprs.fi
2	Aller dans la rubrique "mon compte" afin de récupérer l'APIKEY
3	Sélectionner un indicatif radio (ex. F4MMN)
4	Entrez l'adresse HTTP (Récupérable dans la rubrique API du site https://aprs.fi) suivante sur un navigateur en remplaçant les éléments en gras respectivement par l'indicatif radio souhaité et la clef APIKEY récupéré précédemment: https://api.aprs.fi/api/get?name=OH7RDA&what=loc&apikey=APIKEY&format=json Pour le ciblage multiple il suffit d'ajouter un second indicatif radio en le séparant du précédent via une virgule: https://api.aprs.fi/api/get?name=OH7RDA,OH7AA&what=loc&apikey=APIKEY&format=json Les informations sur la cible (longitude, latitude...) seront transmises en JSON ou en format XML. Pour faire la modification, il faudra changer le paramètre 'format=json' en 'format=xml' D'autres paramètres que la position sont disponibles avec les données de la météo ou envoyer des messages à l'indicatif. Météo: Remplacer le paramètre 'what=loc' par 'what=wx' Messages textuels: Remplacer le paramètre 'what=loc' par 'what=msg'
5	Exemple de retour de l'API via la requête HTTPS/GET: <pre>{ "command": "get", "result": "ok", "what": "loc", "found": 1, "entries": [{ "name": "OH7RDA", "type": "I", "time": "1267445689", "lasttime": "1270580127", "lat": "63.06717", "lng": "27.66050", "symbol": "V#", "srccall": "OH7RDA", "dstcall": "APND12", "phg": "44603", "comment": "VR,W,Wn,Tn Siilinjarvi", "path": "WIDE2-2,qAR,OH7AA" }] }</pre>

Étape	Résultat
6	Paramètres: Description des champs communs • command : La commande API qui a été appelée. • what : L'élément sur lequel portait la requête. • result : Le résultat de la requête, soit « ok », soit « fail » (échec). • found : Le nombre d'entrées renvoyées. Description des champs d'enregistrement de localisation • class : Classe de l'identifiant de la station (a : APRS, i : AIS, w : Web...). • name : Nom de la station, de l'objet, de l'élément ou du navire. • showname : Nom affiché de la station (peut différer du nom unique). • type : Type de cible : « a » pour AIS, « l » pour station APRS, « i » pour élément APRS, « o » pour objet APRS, « w » pour station météo. • time : L'heure à laquelle la cible a signalé cette position (actuelle) pour la première fois (heure d'arrivée aux coordonnées actuelles). • lasttime : L'heure à laquelle la cible a signalé cette position (actuelle) pour la dernière fois. • lat : Latitude en degrés décimaux, le nord est positif. • lng : Longitude en degrés décimaux, l'est est positif. • course : Route sur fond / COG, en degrés. • speed : Vitesse, en kilomètres par heure. • altitude : Altitude, en mètres. • symbol : Table et code des symboles APRS. • srccall : Indicatif d'origine (soit l'indicatif source APRS, soit l'indicatif du navire AIS). • dstcall : Indicatif de destination du paquet APRS. • comment : Commentaire APRS ou destination AIS et heure d'arrivée estimée. • path : Chemin du paquet APRS ou AIS. • phg : Valeur PHG de l'APRS (Puissance-Hauteur-Gain). • status : Dernier message d'état transmis par la station. • status_lasttime : L'heure à laquelle le dernier message d'état a été reçu. Champs additionnels pour les cibles AIS • mmsi : Numéro MMSI du navire AIS. • imo : Numéro IMO du navire AIS. • vesselclass : Code de la classe du navire AIS. • navstat : Code du statut de navigation AIS. • heading : Cap (direction de l'étrave). • length : Longueur du navire AIS en mètres. • width : Largeur du navire AIS en mètres. • draught : Tirant d'eau du navire AIS en mètres. • ref_front : Distance de référence de la position du navire AIS par rapport à l'avant. • ref_left : Distance de référence de la position du navire AIS par rapport à la gauche.



	Action	Description
1	Inclure Leaflet	Ajouter la librairie Leaflet CSS + JS dans le <code><head></code> de ton HTML
2	Créer le conteneur	Ajouter une <code><div id="map"></code> qui accueillera la carte
3	Définir le style	Donner une hauteur au conteneur via CSS (sinon la carte ne s'affiche pas)
4	Initialiser la carte	Créer l'objet <code>L.map()</code> et définir la position + le zoom
5	Charger OpenStreetMap	Ajouter les tuiles OSM avec <code>L.tileLayer()</code>
6	Afficher la carte	Appeler <code>.addTo(map)</code> pour rendre la carte visible

Les structures JSON employées pour la communication inter-composants (par exemple, de l'objet connecté vers le serveur ou de l'API vers l'interface utilisateur) seront spécifiées dans cette section.

```
{
  "command": "get",
  "result": "ok",
  "what": "loc",
  "found": 1,
  "entries": [
    {
      "name": "OH7RDA",
      "type": "I",
      "time": "1267445689",
      "lasttime": "1270580127",
      "lat": "63.06717",
      "lng": "27.66050",
      "symbol": "V#",
      "srcall": "OH7RDA",
      "dstcall": "APND12",
      "phg": "44603",
      "comment": "VR,W,Wn,Tn Siilinjarvi",
      "path": "WIDE2-2,qAR,OH7AA"
    }
  ]
}
```

Gestion des erreurs : Stratégies de reprise et validation des données.		
Scénario d'Erreur Critique	Validation des Données/Détection	Stratégie de Reprise/Action
Perte de Liaison Radio LoRa (Télémétrie)	Absence de réception d'une trame APRS dans un délai prédéfini (Timeout). Vérification du champ RSSI/SNR pour un signal trop faible.	Embarqué (Ballon): Mise en mémoire tampon des données (CSV) sur la carte SD. Tentatives d'émission périodiques (période de retransmission augmentée). Sol: Affichage d'une alerte "Signal Perdu" et utilisation du dernier point de position APRS.fi/OSM connu pour l'estimation.
Détection d'Éclatement/Atterrissage Manquée	Absence de détection (par la Machine à États) alors que l'altitude est significativement basse (pour l'atterrissage) ou la vitesse verticale est anormale (pour l'éclatement).	Sol: Utiliser l'analyse de position APRS.fi et les données de pression/altitude brutes pour une confirmation manuelle par l'équipe au sol.
Les capteurs ne transmettent plus de données	Aucune nouvelle valeur de capteur n'est reçue	Redémarrer la raspberry pi.
Erreur d'écriture sur la carte SD	La raspberry pi est incapable d'écrire des données sur la carte SD	Augmenter la fréquence d'envoi de la télémétrie à 20 secondes au lieu d'une minute
APRS.fi ne répond pas	Si result = 'fail'	Renvoyer une requête / revoir la requête HTTP et corriger s'il y a une erreur.

- Utiliser un diagramme d'activité UML pour illustrer les échanges entre sous-systèmes via ces protocoles.

5. Adaptation des diagrammes de séquence

5.1. Diagrammes de séquence :

- Intégration des choix technologiques : Ajout des interactions concrètes entre les modules logiciels (ex. QTcpSocket, QSerialPort).
- Illustration des échanges sous forme temporelle : Diagrammes réalisés avec Modelio pour refléter les flux de données.



5.2. Diagramme de classes :

- Mise à jour des classes : Ajout des classes issues des frameworks et bibliothèques utilisées.

on avait pas un truc pour ça ?

- Validation de la cohérence : Comparaison avec les cas d'utilisation pour garantir l'alignement.